instituto superior de engenharia de lisboa

LMATE

Programação

Inverno 2026-2027

Trabalho 5

1

Valores e tipos de valores

- Nos programas que vimos até agora já apareceram vários valores. `'Hello World'` é um valor. `40` é outro valor. `0.25` é outro valor. E `True` é outro valor. Outros existem.
- Em Python todos os valores são de algum tipo. O valor `'Hello World'` é do tipo `str`. `40` é do tipo `int`. `0.25` é do tipo `float`. E `True` é do tipo `bool`.
- **As operações que podem ser executadas sobre cada valor estão associadas ao seu tipo.** Por exemplo, é possível dividir um número por outro. Mas, como já vimos nos exemplos de operadores aritméticos, não é possível dividir a *string* `hello` pela *string* `world`. Qual seria o resultado? Não faz nenhum sentido. Não se define. Não é uma instrução Python válida. Dá erro.

Strings

- As *strings* são sequências de caracteres (por exemlo, frases ou textos). Os caracteres podem ser letras, algarismos, ou quaisquer outros como espaços ou mesmo fins de linha.
- O nome do tipo é `str`. São as três primeiras letras da palavra inglesa *string*.
- Para se definir *strings* é necessário usar-se um limitador. Para se definir o princípio e o fim das *strings*.
- O limitador pode ser aspas `"` ou pelicas `'`, ou aspas triplas `"""`, ou pelicas triplas `'''`. Uma *string limitada por aspas pode ter pelicas. Uma* string* limitada por pelicas pode ter aspas. Se a string não tiver aspas nem pelicas, a escolha entre usar aspas ou pelicas como limitador é indiferente.
- A diferença entre aspas ou pelicas, simples e triplas, é que quando se usam triplas o texto pode ocupar várias linhas. É muito útil para textos grandes.
- No teclado portugnês como as aspas só estão acessíveis através da combinação *shift* + `"`, como se fosse uma letra maiúscula, e como as pelicas estão acessíveis diretamente na tecla do ponto de interrogação, sugere-se que se usem as pelicas. É mais rápido.

Exemplos (no ficheiro `exemplos_strings.py`)

```
# uma string definida com aspas
print("uma string")
# uma string definida com pelicas
print('#####') # um separador
# outra string definida com pelicas
print('outra string')
print('#####') # um separador
# as strings entre aspas podem ter pelicas
print("esta string tem pelicas aqui ' e aqui '")
print('#####') # um separador
# as strings entre pelicas podem ter aspas
print('esta string tem aspas aqui " e aqui "')
print('#####') # um separador
# uma string que ocupa 2 linhas definida com aspas
print("""esta é a primeira linha
e esta é a segunda""")
print('#####') # um separador
# uma string que ocupa 3 linhas definida com pelicas
print('''esta é a primeira linha
e esta é a segunda
e esta é a terceira''')
```

Output

```
uma string
#####
outra string
#####
esta string tem pelicas aqui ' e aqui '
#####
esta string tem aspas aqui " e aqui "
#####
esta é a primeira linha
e esta é a segunda
#####
esta é a primeira linha
e esta é a segunda
e esta é a terceira
```

Concatenação de *strings*

- Uma *string* pode ser acrescentada a outra, criado-se uma terceira *string*. A esta operação chama-se concatenação de *strings*. O operador de concatenação é o sinal mais `+`.

Exemplo (no ficheiro `concatenacao_strings.py`)

```
print("esta string" + 'foi concatenada com esta")
```

Output

```
"esta stringfoi concatenada com esta"
```

Observação

- A concatenação de *strings* não acrescenta nada às *strings* concatenadas. Nenhum separador. Concatena as *strings* tal como elas são.

Porquê os limitadores de *string*?

- As *strings* podem conter texto arbitrário. Para que o interpretador Python possa distinguir as *strings* do próprio código Python, é necessário usar-se um limitador para definir o princípio e o fim das *strings*.

Exemplo

- Pretende-se mostrar no *output* o texto "10 + 15 + 7". Usa-se a função `print`. Se não se usar um limitador de *string* o interpretador Python calcula `10 + 15 + 7` e faz *output* do valor 32. Se se usar o limitador de *string* o *output* é o texto "10 + 15 + 7", que é o pretendido. A distinção entre as duas situações só é possível se se usar um limitador de *string*. É por isso que os limitadores de *string* existem.

```
print(10 + 15 + 7)   # o output é 32, não é o pretendido
print('10 + 15 + 7') # o output é 10 + 15 + 7, é o pretendido
```

Strings com aspas e pelicas

- Na eventualidade de um texto conter aspas e pelicas como fazer? Usar as aspas ou as pelicas do texto precedidas do caracter de *escape*. O caracter de *escape* é a barra ao contrário, isto é, a barra contrária à da divisão. Em inglês é o *backslash*. Usa-se o termo *escape* porque se está a escapar ao significado normal da letra. Neste caso está a escapar-se ao significado das aspas e pelicas como limitadores de *string*.

```
print('esta string tem aspas aqui " e pelicas aqui \'')
print("esta string tem aspas aqui \" e pelicas aqui '")
```

- Só por curiosidade, como é que se faz para incluir o *backslash* numa *string*? Faz-se *escape* dele próprio! :-)

```
print('esta string tem o backslash aqui \\')
```

Números inteiros

- Os números inteiros são do tipo `int`. As três primeiras letras da palavra inglesa *integer*.

Exemplos (no ficheiro `exemplos_ints.py`)

Escreva e execute este programa. Confirme o *output*.

```
print(100)
print(-10)
print(0)
print(123)
# valor número inteiro com sinal representável com 64 bits (num CPU moderno)
print(2**63-1)
# e agora um número que ultrapassa o limite dos 64 bits. em Python não dá erro
print(2**200)
```

Output

```
100
-10
0
123
9223372036854775807
1606938044258990275541962092341162602522202993782792835301376
```

Curiosidade

- Em geral a gama de números inteiros disponível em computador está limitado pela arquitetura do CPU. Em geral, nos CPUs modernos, estão disponíveis 64 bits para representar números inteiros.
- A linguagem Python não usa o CPU para representar números inteiros. Por isso a gama de números inteiros não é limitada pelo CPU, mas pela memória RAM do computador. Como a memória dos computadores modernos é da ordem dos Giga Bytes a gama de números inteiros em Python é praticamente ilimitada.

Números decimais

- Os números decimais são do tipo `float`. `float` são as letras iniciais de *floating point*, que quer dizer vírgula flutuande.
- *Floating point* é o formato mais usado em computador para representar números decimais.
- Está disponível a notação científica. `3.5e3` é equivalente a `3.5 vezes (10 elevado a 3)`.

Exemplos (no ficheiro `exemplos_floats.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
print(0.0001)
print(20.1)
print(0.0)
print(-32.35)
print(2e3)  # 2 * (10 elevado a 3)
print(2e-3) # 2 * (10 elevado a -3)
```

Output

```
0.0001
20.1
0.0
-32.35
2000.0
0.002
```

Observação

- O separdor decimal em Pyhton é o `.`, ponto (usado na maioria dos países de língua inglesa). Recorde-se que o separador decimal em Portugal é a `,`, vírgula. A vígula não é válida como separa decimal em Python.

Valores Booleanos

- Em Python há um tipo para valores booleanos. É o tipo `bool`, que são as primeiras letras da palavra inglesa *boolean*. Os valores booleanos ou são "Verdadeiro" ou são "Falso". O valor "Verdadeiro" é `True`. O valor "Falso" é `False` (ambos com a primeira letra maiúscula).

Exemplos (no ficheiro `exemplos_bool.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
print(True)
print(False)
```

Output

```
True
False
```

Curiosidade

- A palavra booleano, ou *boolean* (em inglês), tem origem no nome do matemático inglês George Boole, que formalizou a chamada álgebra de Boole, onde só existem dois valores possíveis, "Verdadeiro" e "Falso".

O Valor Nada - None

- Em Pyhton há um tipo para "nada". O valor "nada" é `None`. O tipo do valor `None` é `NoneType`.

Exemplo (no ficheiro `exemplos_none.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
print(None)
```

Output

```
None
```

Para fazer

Considere o programa Python 3, que se segue.

```
print("hello world")
print('hello world')
print('hello' + 'world')
print("'hello world'")
print("hello world")
print('''hello
world''')
print("""hello
world""")
```

Confirme que o *output* do programa é o que se segue.

```
hello world
hello world
helloworld
'hello world'
hello world
'hello
world'
'hello
world'
```

Indique se as afirmações que se seguem são verdadeiras ou falsas.

1.1

A execução de

```
print('''xJCW2BRUenRSKx2PYh
MVyJUMIaPQga3nFm2''')
```

não dá erro.

1.2

A execução de

```
print('xJCW2BRUenRSKx2PYh"MVyJUMIaPQga3nFm2')
```

não dá erro.

1.3

A execução de

```
print("xJCW2BRUenRSKx2PYh"MVyJUMIaPQga3nFm2")
```

não dá erro.

1.4

A execução de

```
print("xJCW2BRUenRSKx2PYh MVyJUMIaPQga3nFm2")
```

e

```
print("xJCW2BRUenRSKx2PYh" + "MVyJUMIaPQga3nFm2")
```

não dá o mesmo *output*.

1.5

A execução de

```
print('xJCW2BRUenRSKx2PYh
MVyJUMIaPQga3nFm2')
```

dá erro.